Tecnológico de Costa Rica

Curso:
Verificación funcional de circuitos integrados GR 1

Proyecto 1:
Ambiente de Verificación para una FIFO genérica

Profesor:
Ronny Giovanni Garcia Ramirez

Estudiantes:
Gael Agüero Carrillo
Kendy Raquel Arias Ortiz

Fecha:
28 de Abril de 2026

1. Introducción

El presente proyecto consiste en la modificación de un ambiente de verificación de ejemplo visto en clase, aplicado a una FIFO genérica. Se incorpora un esquema de aleatorización controlada en capas, siguiendo un modelo estándar de verificación basado en transacciones.

Adicionalmente, el entorno es extendido para permitir la configuración dinámica del test mediante el uso de plusargs, los cuales modifican las restricciones (*constraints*) de la aleatorización. Esto permite la generación de escenarios dirigidos y casos de esquina (*corner cases*) del diseño.

2. Objetivos

Objetivo General:

Modificar un entorno de verificación para una FIFO genérica, incorporando la aleatorización controlada en capas bajo un modelo estándar de verificación, con soporte de configuración dinámica mediante plusargs.

Objetivos Específicos:

- **a.** Modificar el test base para permitir la utilización de plusargs, los cuales ajustan dinámicamente las restricciones de aleatorización (*constraints*) durante la simulación.
- **b.** Permitir la generación de casos de esquina (corner cases) mediante la variación controlada de los parámetros de entrada del entorno.
- **c.** Documentar de forma detallada el diseño del plan de pruebas, incluyendo arquitectura del entorno de verificación, diagramas del sistema, descripción de cada componente del testbench, y explicación de los casos de prueba implementados.
- **d.** Gestionar el desarrollo del proyecto mediante control de versiones en GitHub, asegurando trazabilidad y control del código fuente.
- **e.** Validar la correcta ejecución del entorno en el servidor de simulación del laboratorio de electrónica.

3. Arquitectura del Entorno

3.1. Estrategia de Solución y Desarrollo del Proyecto

El desarrollo del proyecto se realizó a partir de la separación y estructuración del código base en módulos funcionales, siguiendo una arquitectura por capas inspirada en el modelo estándar de verificación.

Como parte de esta reorganización, se introdujo un nuevo componente de observación denominado monitor, encargado de capturar las señales del DUT y facilitar su análisis dentro del entorno de verificación. Los demás módulos fueron adaptados para cumplir con sus responsabilidades específicas dentro del flujo de verificación. Posteriormente, se habilitó el uso de plusargs en el test base, permitiendo la configuración dinámica de parámetros de simulación y la modificación de restricciones de aleatorización sin necesidad de recompilación.

Adicionalmente, se estableció la correcta conexión entre el sistema de verificación y el monitor, asegurando la coherencia del flujo de transacciones y evitando errores de simulación durante la ejecución. Como resultado, el entorno final está compuesto por 10 módulos claramente definidos, cada uno con responsabilidades específicas dentro de la arquitectura de verificación.

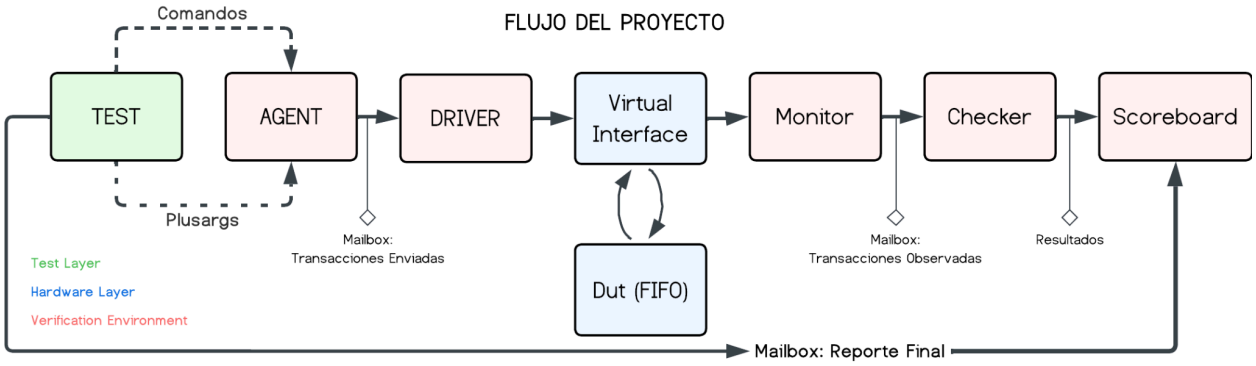

Imagen 1. Diagrama del Flujo entre los Módulos del Proyecto Final

3.2. Descripción de los Módulos del Entorno (Comparación y Cambios Actuales)

A - Módulo fifo.sv

La unidad bajo prueba es una Memoria FIFO Sincrónica, un componente de infraestructura digital diseñado para el desacoplamiento de flujos de datos. Su propósito fundamental en este proyecto es actuar como un regulador de tráfico (buffer) que permite la transferencia de información entre un productor (Agente/Driver) y un consumidor (Monitor/Checker). La relevancia de este módulo radica en su capacidad para gestionar la latencia y las diferencias de velocidad en el procesamiento de datos, garantizando que el orden de salida sea idéntico al de entrada sin introducir corrupción de bits ni colisiones en el bus de datos.

El comportamiento operativo de la FIFO se rige por una Finitura de Estados determinada por su capacidad física (depth) y su ancho de palabra (bits). Lógicamente, el dispositivo opera mediante una administración estricta de direcciones a través de dos mecanismos: el puntero de escritura, que señala la ubicación disponible para el próximo dato entrante mediante la señal push, y el puntero de lectura, que rastrea la ubicación del dato más antiguo para ser extraído mediante la señal pop. La correcta sincronización de estos punteros con el flanco de subida del reloj (clk) es lo que define la estabilidad del sistema, especialmente en escenarios de alta demanda donde ocurren operaciones simultáneas de lectura y escritura.

Un aspecto importante de la FIFO en este proyecto es su lógica de protección y monitoreo de estado, representada por las banderas de control "*full*" y "*pndng*". Estas señales no son meramente informativas; son mecanismos de retroalimentación esenciales para el control de flujo del hardware. La señal *full* actúa como un freno de seguridad que impide el desbordamiento (overflow) cuando la memoria alcanza su capacidad máxima, mientras que la señal *pndng* indica la presencia de datos válidos para el consumidor. La verificación se centra precisamente en validar que estas banderas se activen y desactiven en el ciclo de reloj exacto, evitando condiciones de carrera o sub desbordamientos (underflow) que comprometerían la integridad del sistema digital completo.

A.1 Análisis Comparativo de las Implementaciones de Hardware

Dentro del proyecto se han integrado dos filosofías de diseño distintas para el mismo propósito funcional. La primera, *"fifo_flops"*, utiliza una arquitectura de registros en cascada donde los datos se desplazan físicamente de una etapa a otra como en un registro de desplazamiento. Este diseño es ideal para aplicaciones de baja latencia pero alta complejidad en la lógica combinacional de salida, ya que requiere una red de multiplexores para seleccionar el dato de salida. Por el contrario, la versión *"fifo_generic"* implementa un buffer circular basado en un arreglo de memoria estática. En este caso, el dato permanece inmóvil y la lógica se centra en el manejo aritmético de los punteros, una técnica más eficiente en términos de área de silicio para profundidades de almacenamiento mayores.

La inclusión de estas dos variantes justifica la necesidad de un entorno de verificación basado en SystemVerilog y Programación Orientada a Objetos. Al tener comportamientos internos tan dispares (uno basado en el movimiento físico de datos y el otro en la gestión de índices), se pone a prueba la versatilidad del testbench. El entorno debe ser capaz de abstraer estas diferencias y verificar que, ante las mismas secuencias de estímulos, ambos módulos produzcan resultados idénticos y respeten el protocolo de comunicación, demostrando así que la lógica de verificación es independiente de la arquitectura física del hardware.

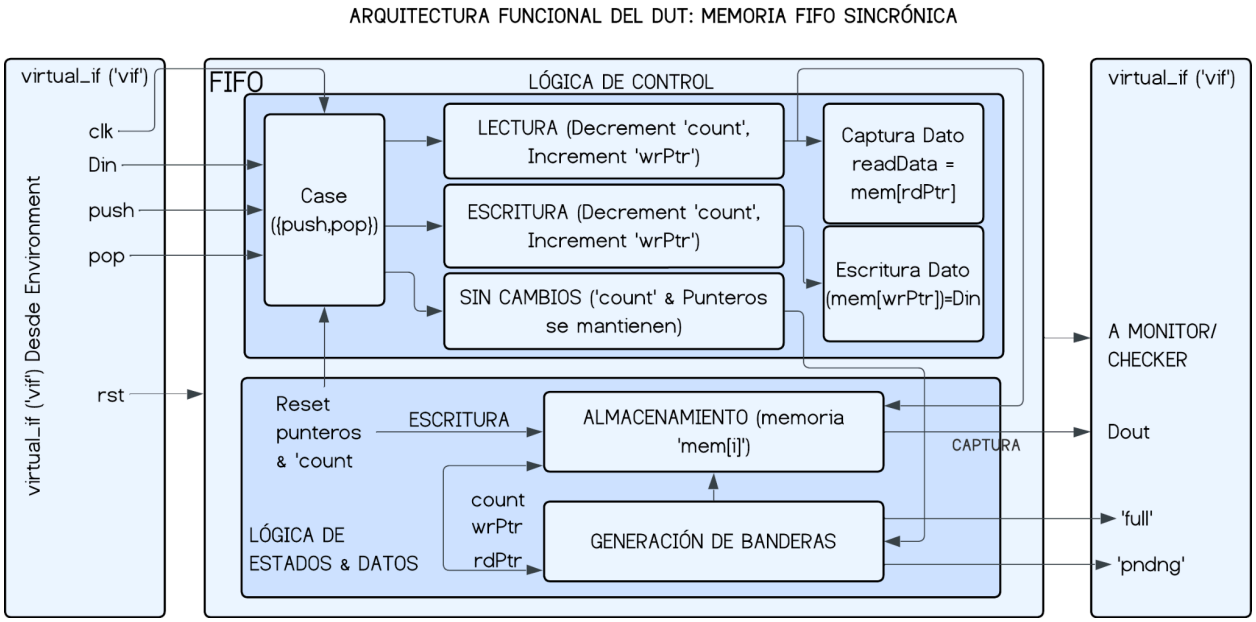

Imagen 2. Arquitectura Funcional de la FIFO (DUT)

B - Módulo test_bench.sv

El módulo test_bench.sv constituye el nivel superior de la jerarquía de verificación, donde se integran el hardware (DUT) y el software de prueba. En su versión inicial, el código presentaba una estructura simplificada con parámetros de ancho (width) y profundidad (depth) estáticos, fijados directamente en el código fuente. En términos de conectividad, la versión inicial realizaba un enlace parcial de las interfaces virtuales, conectando únicamente el componente del Test y el Driver, lo que limitaba la capacidad de observación del sistema al no asegurar explícitamente el enlace del Monitor desde el nivel superior.

La versión final del test_bench.sv introduce una mejora sustancial en la flexibilidad operativa mediante el uso de directivas de compilación y macros condicionales (ifndef WIDTH). Esta implementación permite que el banco de pruebas sea parametrizable desde la línea de comandos de la terminal, eliminando la necesidad de modificar el código fuente para probar diferentes configuraciones de hardware. Esta capacidad de configuración dinámica es un avance valioso para la escalabilidad, permitiendo que el mismo archivo sea utilizado en diversas etapas del diseño simplemente sobrescribiendo los valores de los parámetros durante la fase de compilación.

En cuanto a la infraestructura de software, la versión final presenta una gestión de dependencias mucho más robusta y organizada. Se observa la inclusión explícita de componentes como el monitor.sv y que no estaba presente en todas las versiones preliminares, asegurando que el entorno de verificación esté completo. Además, se añadió un bloque de diagnóstico inicial en el *"initial begin"* que imprime un cuadro de configuración en la consola del simulador. Este detalle, aunque sencillo, es fundamental en entornos profesionales para verificar rápidamente que los parámetros de simulación coincidan con los objetivos de la prueba antes de procesar grandes volúmenes de datos.

Finalmente, el cambio más significativo en la versión final radica en el mapeo jerárquico de interfaces virtuales. Mientras que la versión inicial se limitaba a conectar el Driver, la versión final propaga la interfaz física *"_if"* sistemáticamente hacia la instancia del ambiente, el driver y el monitor: *"t0.ambiente_inst.monitor_inst.vif = _if"*. Esta conexión total garantiza que el monitor tenga visibilidad completa de los puertos de la FIFO, permitiendo una captura de datos precisa para el Scoreboard. Asimismo, se conservó y refinó el mecanismo de Watchdog basado en el tiempo de simulación, asegurando que cualquier fallo lógico que cause un bloqueo en el hardware no resulte en un consumo infinito de recursos de cómputo, finalizando la prueba de manera segura al alcanzar el límite de tiempo establecido.

Descripción del Flujo de Inicialización:

1. **Definición de Parámetros:** El sistema evalúa si se pasaron macros desde la terminal (ej. +define+WIDTH=32) o si debe usar los valores por defecto (16 bits, 8 de profundidad).
2. **Instanciación del Hardware (DUT):** Se crea la *"fifo_generic"* y se conectan sus puertos físicos a las señales de la interfaz _if.
3. **Configuración del Entorno de Software:** Se crea el objeto t0 (Test). En este punto, se imprime en consola el cuadro de "Configuración del Testbench".
4. **Enlace de Interfaces Virtuales:** Es la fase de "cableado virtual". Se asigna la interfaz física a tres puntos críticos:
- Al cerebro del test (t0._if).
- Al Driver (driver_inst.vif), para que pueda inyectar datos.
- Al Monitor (monitor_inst.vif), para que pueda observar las salidas.
5. **Arranque de Procesos:** Se inicia la generación del reloj y se dispara el task run del test en segundo plano, activando simultáneamente el temporizador de seguridad (Watchdog).

Como se observa en el Diagrama del Flujo de Inicialización (Imagen 3), el Testbench opera como el director de orquesta del sistema. El flujo garantiza que los parámetros de hardware se definan primero, permitiendo que la instanciación sea dinámica. El punto esencial de este proceso es la Conexión de Interfaces Virtuales, ya que asegura que, una vez iniciada la simulación, tanto el Generador de Reloj como el Watchdog y la Secuencia del Test operen en paralelo y con visibilidad total sobre las señales físicas del DUT.

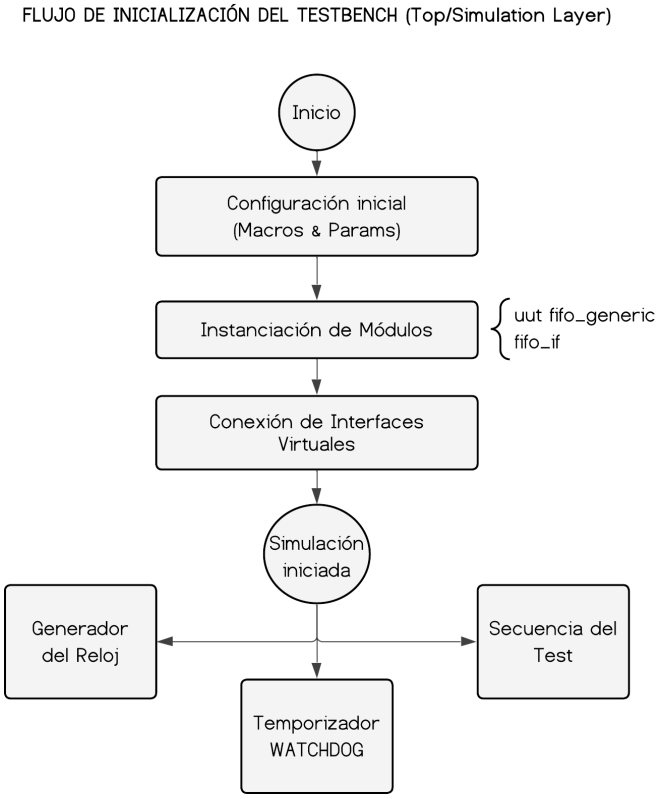

Imagen 3. Diagrama del Flujo de Inicialización del Testbench

C - Módulo test.sv

El módulo test.sv representa la capa de inteligencia y orquestación del entorno. En su versión inicial, el componente operaba bajo un esquema de secuenciamiento rígido, donde las pruebas consistían en cuatro pasos fijos (llenado, transacción única, específica y secuencia) con parámetros como *"num_transacciones"* y *"max_retardo"* grabados estáticamente en el código. Esta arquitectura limitaba la verificación a un único escenario predecible, obligando al ingeniero a modificar y recompilar el archivo cada vez que se deseaba cambiar la intensidad o el tipo de tráfico, lo que resultaba ineficiente para detectar errores complejos o condiciones de carrera aleatorias.

La versión final del código transforma grandemente este enfoque mediante la implementación de configuración dinámica por Plusargs y aleatoriedad pesada. El nuevo módulo utiliza la función "*$value$plusargs"* para extraer parámetros directamente de la terminal en tiempo de ejecución, permitiendo modificar desde la semilla aleatoria (SEMILLA) hasta la cantidad de transacciones (NUM_TRANS) sin tocar una sola línea de código. Además, se introdujo una tabla de visualización estética al arranque que resume la configuración del test, facilitando la trazabilidad de los experimentos de verificación y asegurando que los parámetros de hardware y software coincidan antes de procesar el tráfico.

Una de las mejoras técnicas más notables es la inclusión de la tarea "*estado_inicial()*". Mientras que la versión original siempre comenzaba con la FIFO vacía, la versión final permite forzar estados de inicio específicos como LLENO, MITAD o VACÍO mediante comandos externos. Esto es muy importante para verificar los Corner Cases (casos de esquina), permitiendo comprobar, por ejemplo, si la FIFO gestiona correctamente un push cuando ya está llena desde el primer ciclo de reloj. Asimismo, el sistema de

generación de estímulos evolucionó hacia un modelo basado en pesos porcentuales (PESO_L, PESO_E, PESO_S, PESO_R), lo que permite configurar la prueba para que sea puramente de lectura, puramente de escritura o un estrés de operaciones simultáneas y resets inesperados.

Finalmente, el flujo de ejecución en el task run se profesionalizó mediante el uso de sincronización avanzada y reportes automáticos. En lugar de un tiempo de espera fijo de #10000, la versión final calcula dinámicamente el tiempo de espera necesario basado en el número de transacciones y el retardo máximo configurado. Al concluir el tráfico, el test coordina de forma determinista la solicitud de métricas de rendimiento y reportes de errores al Scoreboard, cerrando la simulación con una marca de tiempo precisa. Esta evolución de un modelo estático a uno dirigido por datos incrementa la cobertura de verificación, permitiendo que una misma arquitectura de test sea capaz de encontrar errores en cualquier implementación de FIFO, sin importar su tamaño o complejidad interna.

Descripción del Flujo de Ejecución:

1. **Inicio y Conexión Jerárquica:** El objeto test se instancia y realiza el "cableado virtual" inicial. En esta etapa, asigna la interfaz física "*_if*" a los punteros virtuales (vif) del Driver y el Monitor. Esta conexión sistemática garantiza que todos los componentes de software tengan visibilidad total de las señales del hardware antes de iniciar el tráfico.
2. **Configuración Dinámica (Plusargs):** Mediante la función técnica "*leer_plusargs()*", el sistema extrae parámetros externos desde la terminal del simulador. El código evalúa variables como la semilla aleatoria (SEMILLA), el número de transacciones (NUM_TRANS) y los tiempos de espera. Esta capacidad permite que el mismo banco de pruebas sea utilizado tanto para verificaciones rápidas de humo como para pruebas de regresión masivas, simplemente cambiando los comandos de ejecución.
3. **Preparación del Escenario (estado_inicial):** Antes de la generación de tráfico masivo, el test ejecuta una tarea de configuración de estado. Esta función permite forzar a la FIFO a iniciar en condiciones críticas: LLENA, MITAD o VACÍA. Esto es fundamental para validar los **Casos de Esquina** (Corner Cases), asegurando que el diseño gestione correctamente eventos como un intento de escritura en una memoria que inicia saturada.
4. **Batería de Pruebas por Pesos (Estrés Dirigido):** La versión final utiliza un modelo de aleatoriedad pesada. En lugar de ejecutar pasos fijos, el test configura al Agente con porcentajes de probabilidad (PESO_L, PESO_E, PESO_S, PESO_R). Esto permite dirigir la prueba hacia escenarios específicos de estrés, como por ejemplo, priorizar operaciones simultáneas de lectura y escritura para detectar colisiones de punteros o fallos en las banderas de estado.
5. **Fase de Cierre y Auditoría:** Una vez completada la carga de transacciones, el test calcula un tiempo de espera dinámico proporcional al volumen de datos procesados. Finalmente, el módulo coordina la recolección de métricas de rendimiento, enviando las instrucciones retardo_promedio y reporte al Scoreboard. Esto permite consolidar los resultados de latencia, integridad de datos y errores detectados antes de finalizar formalmente la simulación mediante el comando $finish.

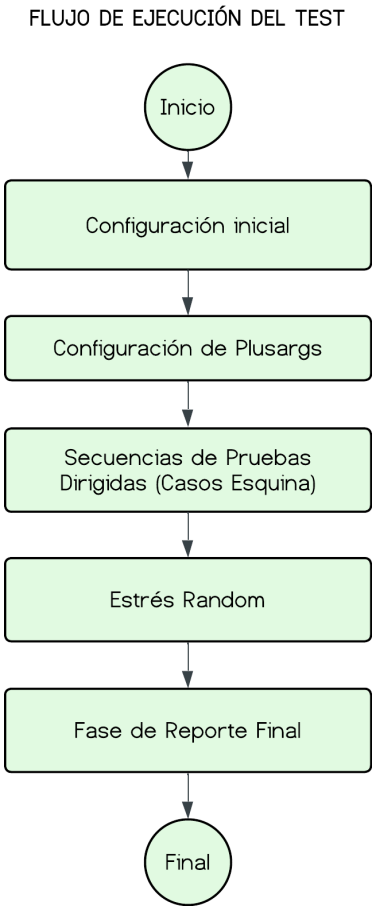

Imagen 4. Diagrama del Flujo de Ejecución del Test

D - Módulo ambiente.sv

El módulo ambiente.sv actúa como el contenedor de nivel superior de la infraestructura de verificación, encargado de instanciar, conectar y coordinar todos los componentes de la arquitectura. Su función principal es encapsular la complejidad del entorno, permitiendo que el objeto Test interactúe con una única entidad para el desarrollo de la simulación.

En la versión original, el ambiente presentaba una arquitectura incompleta con la ausencia de un monitor. Esto obligaba al Driver a enviar los datos "proyectados" directamente al Checker "*(drv_chkr_mbx)"*. Esta estructura era conceptualmente débil, ya que el Checker validaba el diseño basándose en lo que el Driver intentaba inyectar, y no en lo que el hardware (DUT) realmente respondía en sus pines físicos. Además, las conexiones de mailboxes de control estaban rígidamente ancladas en el constructor, limitando la flexibilidad de configuración desde el nivel de prueba.

La versión final del módulo implementa una arquitectura completamente segregada y orientada a objetos. La mejora fundamental es la integración del Monitor, lo que permite separar físicamente la responsabilidad de inyectar estímulos (Driver) de la responsabilidad de observar la respuesta real del hardware. En este nuevo modelo, la comunicación fluye de forma natural y precisa: el Monitor captura las señales de la interfaz "*física _if*" y envía las transacciones reales al Checker a través del nuevo "*mon_chkr_mbx"*.

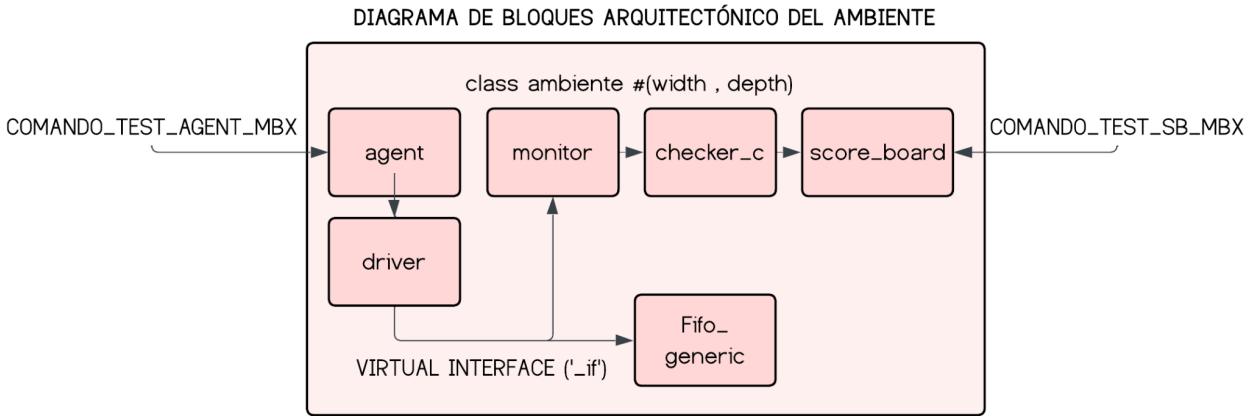

Imagen 5. Diagrama de Bloques Arquitectónico del Ambiente

El diagrama arquitectónico ilustra la jerarquía de la clase ambiente, destacando la independencia del Monitor como observador del hardware. Se observa cómo el flujo de datos para la validación (Monitor → Checker → Scoreboard) es independiente del flujo de inyección (Agent → Driver), garantizando una verificación de "caja negra" objetiva y robusta.

E - Módulo agent.sv

El Agente actúa como el estratega principal del entorno de verificación, funcionando como el motor que impulsa la simulación al decidir qué tipo de tráfico debe enfrentar el hardware. Su responsabilidad principal es la generación de secuencias de estímulos basadas en las directrices enviadas por el Test, operando a un nivel de abstracción superior donde determina el volumen y la naturaleza de los datos antes de entregarlos al Driver. En la implementación final, este componente ha evolucionado hacia una estructura mucho más flexible y potente mediante el uso de variables de configuración dinámicas y una lógica de generación inteligente.

Una de las adiciones más destacadas en esta versión es la capacidad de controlar el flujo mediante parámetros externos o "*plusargs*", como se observa en la variable "*solo_escrituras*". Esta funcionalidad permite modificar el comportamiento del entorno en tiempo de ejecución sin necesidad de recompilar el código, permitiendo, por ejemplo, forzar un llenado constante de la FIFO para estresar las condiciones de saturación de manera inmediata. Además, el Agente incorpora ahora una función de generación de datos que admite patrones específicos como 0xAAAA, 0x5555 o secuencias incrementales, lo cual es fundamental para validar la integridad de los bits y detectar fallos de "bus pegado" en el diseño.

La lógica interna del módulo se basa en un selector de escenarios que permite al ingeniero elegir entre ráfagas de tráfico aleatorio, secuencias dirigidas de llenado y vaciado, o la inyección de transacciones específicas. Esta última capacidad es vital para el proceso de depuración o "debugging", ya que permite definir manualmente datos, retardos y tipos de operación exactos para reproducir casos de esquina detectados previamente. La introducción de un sistema de pesos probabilísticos permite que la generación aleatoria sea dirigida; por ejemplo, se puede configurar el agente para que el 80% de las transacciones sean de escritura, forzando escenarios de estrés que serían difíciles de alcanzar con estímulos puramente manuales.

Finalmente, al estar completamente parametrizado con el ancho de palabra del sistema, el Agente garantiza que cada paquete generado sea siempre coherente con la arquitectura de la FIFO bajo prueba. Esta versión definitiva lo transforma de un generador rígido en un "Jefe de Tráfico" inteligente y

altamente configurable. La separación clara entre la fase de creación del objeto y la de envío al Driver asegura un entorno de simulación profesional y ordenado, facilitando la exploración exhaustiva de los estados del hardware y aumentando significativamente la cobertura de la verificación.

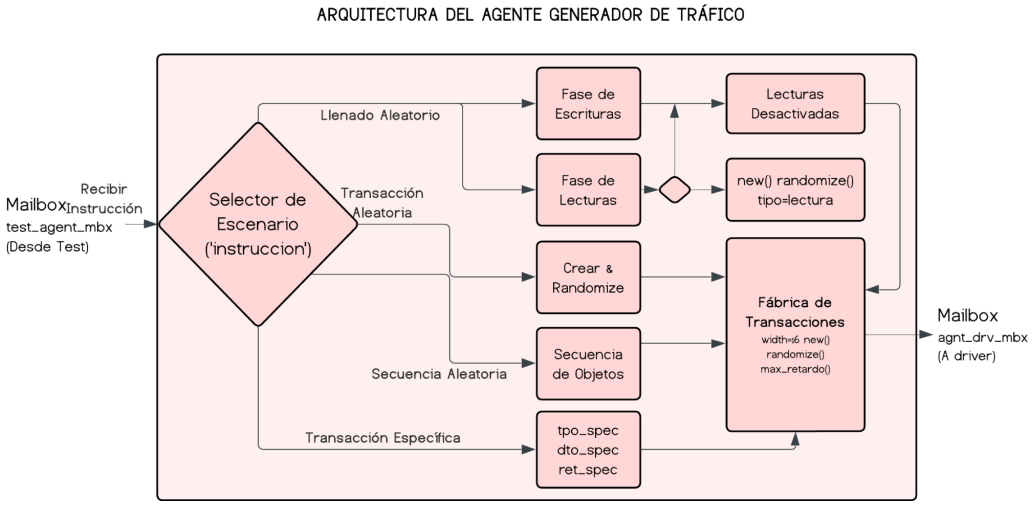

Imagen 6. Diagrama de Bloques Arquitectónico del Generador de Tráfico

Como se aprecia en la Imagen 6, el Agente implementa un selector de escenarios que permite bifurcar la lógica de generación. El diagrama ilustra correctamente la nueva funcionalidad de 'Lecturas Desactivadas' controlada por *"Plusargs"* y la capacidad de inyectar transacciones específicas para la depuración de casos de esquina, consolidando al Agente como el motor de estímulos del testbench.

F - Módulo driver.sv

El Driver actúa como la capa de abstracción física entre el entorno de verificación y el hardware real (DUT). Su responsabilidad principal radica en la "serialización" o traducción de las transacciones abstractas recibidas desde el Agente a señales lógicas reales que la FIFO pueda interpretar. A diferencia del Agente, el Driver opera bajo una sincronización estricta con la señal de reloj de la interfaz virtual (vif.clk), lo que asegura que cada estímulo inyectado respete los tiempos de configuración y mantenimiento indispensables en los sistemas digitales. En la implementación final, se mejoró la gestión de señales al inicio de cada ciclo de ejecución, garantizando que los pines de control (push y pop) regresen siempre a un estado inactivo tras completar una operación. Esta práctica previene errores comunes de simulación, como lecturas o escrituras "fantasma", donde una señal queda activada accidentalmente.

Una de las capacidades más fuertes de esta versión es el soporte para la operación de Escritura y Lectura simultánea. Esta funcionalidad permite ejecutar pruebas de estrés críticas, verificando si la lógica de control de la FIFO es capaz de gestionar un dato entrante y uno saliente en el mismo flanco de reloj sin corromper sus punteros internos ni generar banderas de estado erróneas. El manejo de tiempos también se optimizó para ser más realista, utilizando un contador de retardo basado exclusivamente en ciclos de reloj en lugar de tiempos absolutos, lo que simula con exactitud la latencia propia de una arquitectura de hardware real. Finalmente, el Driver ahora integra una captura de datos más eficiente para las operaciones de lectura. El dato presente en la interfaz se almacena inmediatamente en el objeto de la transacción "*dato_out*", facilitando la trazabilidad de la información que fluye a través del sistema.

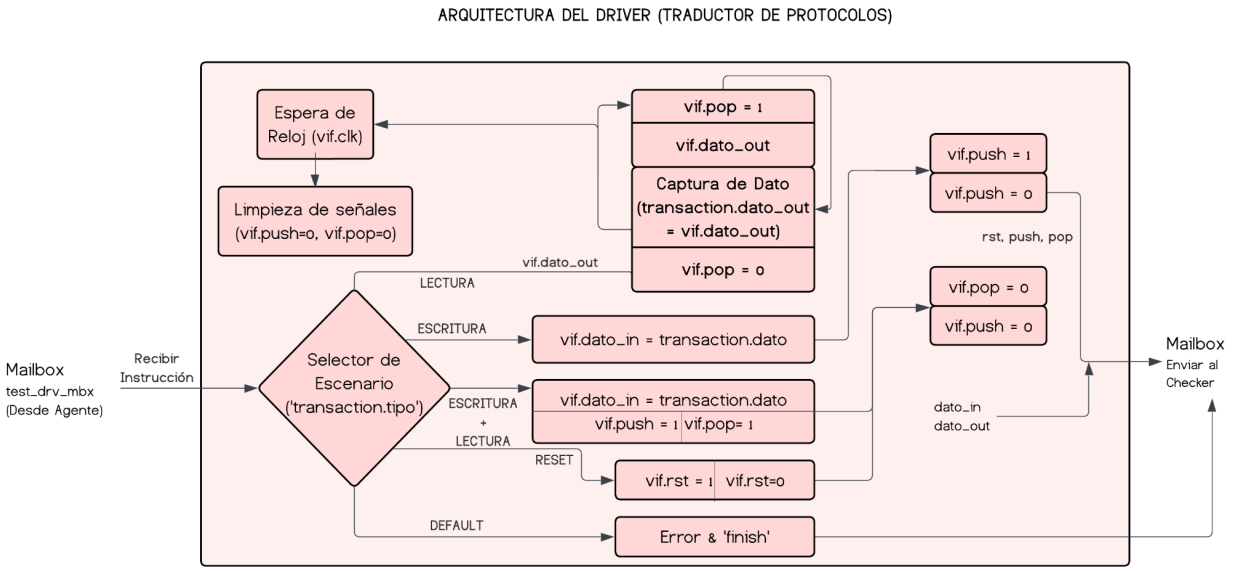

Imagen 7. Diagrama de Bloques Arquitectónico del Driver

El diagrama de bloques de la Imagen 7 ilustra la lógica interna de ejecución del Driver, detallando el selector de escenarios para las operaciones de lectura, escritura, reset y la prueba de estrés simultánea. Cabe destacar que, en la arquitectura final, la salida hacia el Mailbox del Checker se ha desplazado al módulo Monitor para garantizar una verificación de tipo 'caja negra' (Black-box), permitiendo que el Driver se concentre exclusivamente en la inyección de protocolos sincronizados con la interfaz virtual.

G - Módulo monitor.sv

El módulo monitor.sv representa la incorporación más significativa en la evolución de la arquitectura de verificación, actuando como un componente de observación pasiva encargado de capturar la actividad real en los pines de la FIFO. Mientras que el Driver tiene la función de "hablarle" al hardware mediante la inyección de señales, el Monitor se encarga exclusivamente de "escucharlo", abstrayendo el comportamiento físico de los cables y traduciéndolo nuevamente en objetos de transacción que el resto del entorno pueda interpretar. En la versión inicial del proyecto, este componente estaba ausente como entidad independiente, lo que generaba una dependencia poco fiable donde el Checker sólo validaba las intenciones del Driver y no los resultados reales en el hardware.

La implementación final del Monitor utiliza una lógica de muestreo síncrono basada en un ciclo forever que se activa en cada flanco de subida del reloj (posedge vif.clk). Una de sus características técnicas más importantes es la implementación de detección de flancos para el control de flujo. Mediante el uso de variables de estado anterior "*last_push, last_pop, last_rst*", el monitor asegura que solo se genere una transacción hacia el Checker cuando se detecta un cambio de estado de bajo a alto. Este mecanismo de "debounce" lógico es fundamental para evitar la captura redundante de transacciones en ciclos donde una señal de control permanece activa durante varios periodos de reloj, garantizando la integridad de las estadísticas en el Scoreboard.

Además, el código del Monitor establece una jerarquía de prioridades para la captura de eventos mediante una estructura "*if-else*" cuidadosamente diseñada. El sistema otorga prioridad máxima a la detección del Reset, seguida de las operaciones simultáneas de Escritura y Lectura, y finalmente las operaciones individuales. Esta organización asegura que, en escenarios de estrés donde ocurren múltiples eventos en el

mismo flanco de reloj, el monitor sea capaz de reconstruir la escena completa sin perder información crítica. Al capturar el "*dato_out*" directamente de los pines de la interfaz, el Monitor permite que el Checker realice una validación de "caja negra" auténtica, siendo capaz de detectar si la FIFO devolvió un valor incorrecto o si ocurrió una corrupción silenciosa en la memoria interna del diseño.

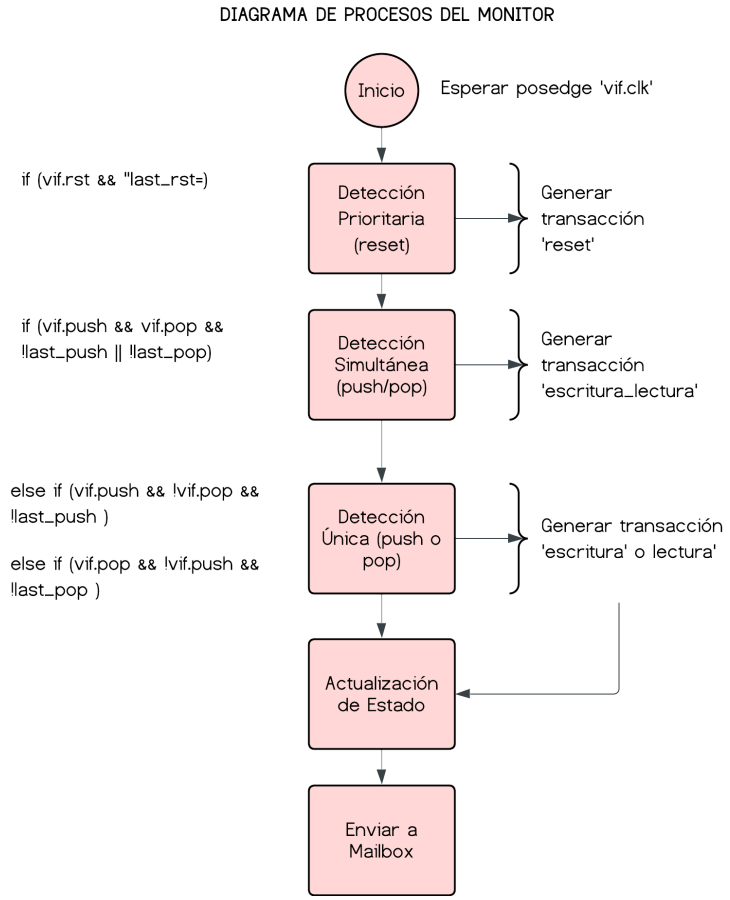

Imagen 8. Diagrama de Procesos del Monitor

El diagrama de procesos del monitor detalla la máquina de estados lógica encargada de la captura de transacciones. Se observa la estructura jerárquica de toma de decisiones, donde el Reset tiene prioridad absoluta, seguido de las operaciones de estrés (simultáneas), asegurando que cada cambio de estado en la interfaz física sea debidamente registrado y enviado al Mailbox del Checker.

H - Módulo checker.sv

El módulo checker.sv actúa como el juez de validez dentro del entorno de verificación. Su responsabilidad fundamental es mantener un Modelo de Referencia (Golden Model) mediante el uso de una cola emulada "*emul_fifo*", la cual replica el comportamiento ideal que debería tener la FIFO. La transición entre la versión original y la definitiva representa un salto cualitativo en la arquitectura; mientras que antes el Checker dependía de los datos enviados por el Driver (validando únicamente las "intenciones" del entorno), el Checker final recibe la información directamente desde el Monitor. Esto garantiza una independencia total, permitiendo validar lo que realmente ocurrió en los pines físicos del hardware.

El código final es la gestiona el desfase de punteros "*ptr_desfase*" ante condiciones de error. El Checker ahora es capaz de emular cómo el hardware real desincroniza su puntero de lectura "*rdPtr*" cuando ocurre un Underflow. Mediante una lógica de resincronización inteligente, el Checker "descarta" escrituras futuras hasta que los punteros del modelo y del hardware vuelven a estar alineados. Esta capacidad de modelar fallos de protocolo sin detener la simulación permite un análisis mucho más profundo que el de la versión original.

Asimismo, el Checker final introduce una lógica avanzada para operaciones de Escritura y Lectura simultánea, contemplando escenarios complejos como el Bypass. Este mecanismo verifica si, cuando la FIFO está vacía, un dato puede entrar y salir en el mismo ciclo de reloj de manera íntegra. Para asegurar la fidelidad del modelo, se implementó el uso de copias profundas "*copy_trans = new()*". A diferencia de la versión inicial, donde se corrían riesgos de corrupción por punteros compartidos, la versión actual garantiza que cada objeto dentro de la "*emul_fifo"* sea una entidad única y protegida, evitando errores de software en el testbench que podrían enmascarar fallos del hardware.

Por último, el sistema de reporte se ha fortalecido mediante el uso de la directiva "*$error*" y una comunicación detallada con el Scoreboard. Durante un evento de Reset, el Checker no sólo limpia su memoria interna, sino que cuantifica y reporta exactamente cuántos datos se perdieron, permitiendo una trazabilidad total del flujo de información. Ante cualquier discrepancia entre el dato capturado por el monitor y el esperado por el modelo, el Checker detiene la simulación y señala la falla exacta, eliminando cualquier posibilidad de errores silenciosos y asegurando que el diseño cumpla completamente con su especificación técnica.

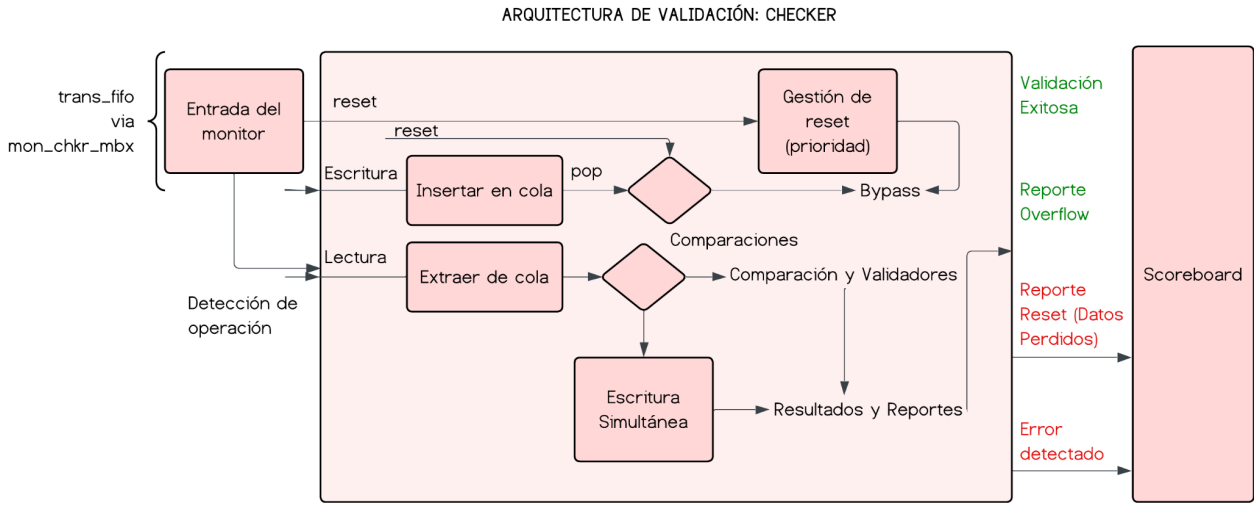

Imagen 9. Arquitectura de Validación del Checker

I - Módulo scoreboard.sv

El módulo score_board.sv actúa como el centro de inteligencia de datos del entorno, recolectando cada veredicto emitido por el Checker para consolidar las métricas de desempeño y la trazabilidad de la FIFO. Su evolución principal radica en la estricta gestión de tipos y parámetros; en la versión final, se asegura que todas las estructuras de datos, como la cola histórica "*scoreboard[$]*" y los punteros auxiliares, respeten el parámetro #(.width(width)). Esta corrección elimina errores de compilación por

desajuste de tamaños y garantiza que los reportes de datos sean coherentes con la configuración física del hardware.

Una mejora considerable en la lógica de reporte es la optimización de la visualización del historial. En la versión original, el proceso de generar un reporte era destructivo e ineficiente, pues extraía los datos de la cola *"pop_front"* y los movía a un arreglo auxiliar para no perderlos. El código definitivo simplifica radicalmente esta tarea mediante el acceso directo por índice *"scoreboard[i].print"*. Este cambio no solo reduce la carga computacional en pruebas de estrés prolongadas con miles de transacciones, sino que elimina el riesgo de corromper la base de datos histórica por errores de punteros durante la simulación.

El Scoreboard implementa una lógica de cálculo de latencia promedio mucho más completa y precisa. Se añadió una validación fundamental para evitar la división por cero y se introdujo la conversión explícita de tipos mediante *"$itor"*. Esto asegura que el cálculo del retardo se realice en el dominio de punto flotante, evitando la pérdida de información por truncamiento de enteros que sufría la versión inicial. Gracias a su capacidad de respuesta a órdenes externas (vía test_sb_mbx), el componente permite generar reportes bajo demanda, proporcionando un resumen claro del comportamiento temporal y la integridad de la FIFO en tiempo real.

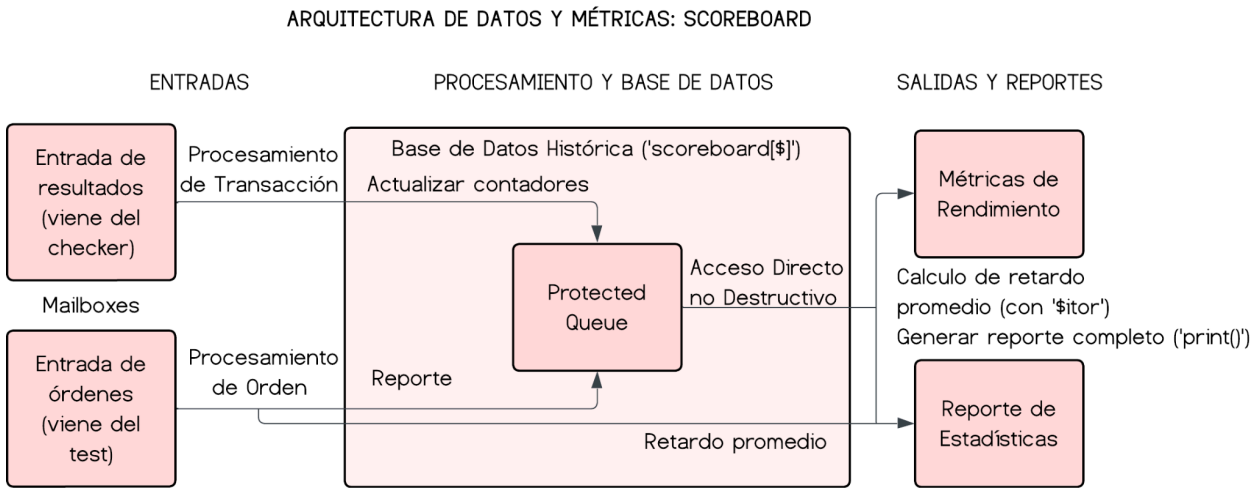

Imagen 10. Arquitectura de Datos y Métricas del Scoreboard

3.3 Estructuras de Datos y Protocolo de Conexión: Definiciones Globales (interface_transactions.sv)

La interfaz "*fifo_if*" constituye el punto de acoplamiento entre el entorno de verificación y el DUT, agrupando en una única abstracción todas las señales físicas necesarias para la operación de la FIFO, tales como *"push, pop, data_in, data_out"*, así como las banderas de estado *"full, empty y pndng"*. Al definirse como una interfaz parametrizada, esta estructura permite adaptar el entorno a diferentes configuraciones de ancho de palabra y profundidad sin necesidad de modificar los componentes del testbench. Sin embargo, el aspecto más relevante de este diseño no radica únicamente en la encapsulación de señales, sino en el uso de una interfaz virtual como mecanismo de conexión entre los componentes de verificación y la instancia física del DUT.

En lugar de acceder directamente a las señales del hardware, módulos como el Driver y el Monitor operan sobre una referencia abstracta de la interfaz, lo que introduce un nivel de desacoplamiento fundamental.

Este enfoque permite que dichos componentes sean independientes de la instancia concreta del DUT, facilitando su reutilización, mantenimiento y portabilidad a otros entornos de verificación.

En este contexto, el Driver utiliza la interfaz virtual para traducir las transacciones de alto nivel generadas por el Agente en señales físicas sincronizadas con el reloj, implementando así la inyección de estímulos sobre el DUT. De forma complementaria, el Monitor observa estas mismas señales a través de la interfaz, reconstruyendo las transacciones efectivamente ejecutadas por el hardware. Esta dualidad garantiza que tanto la generación como la observación de eventos se realicen sobre una base común, eliminando inconsistencias y permitiendo una verificación coherente del comportamiento del sistema.

De esta manera, la interfaz virtual no solo actúa como un puente de conexión, sino como un elemento estructural que define la arquitectura del entorno de verificación, habilitando un flujo de información claro entre generación, ejecución y análisis. Este patrón es consistente con metodologías modernas de verificación basadas en componentes, donde el desacoplamiento entre estímulo y hardware es un requisito esencial para la construcción de entornos robustos y escalables.

4. Plan de Pruebas y Estrategia de Validación

La validación del diseño se fundamenta en una estrategia de Verificación Basada en Restricciones (CRV), complementada con la ejecución de regresiones automatizadas que permiten explorar de manera sistemática el espacio de estados del sistema. A diferencia de un enfoque tradicional basado en vectores de prueba estáticos, este entorno genera estímulos dinámicos mediante aleatorización controlada, lo que posibilita la detección de condiciones de borde y comportamientos no triviales que difícilmente serían identificados de forma manual. El proceso de validación es ejecutado por el script *"run_fifo_regression.sh"*, el cual automatiza tanto la compilación como la ejecución de múltiples instancias de simulación. Una característica clave de este flujo es que el diseño se compila una única vez utilizando configuraciones máximas de ancho y profundidad, mientras que las variaciones específicas de cada prueba se introducen en tiempo de ejecución mediante el uso de "plusargs". Este enfoque reduce significativamente el tiempo de simulación y permite una alta flexibilidad en la exploración del espacio de pruebas.

Cada iteración de la regresión define de forma pseudoaleatoria tanto los parámetros estructurales del DUT como las condiciones operativas de la simulación. Entre estos parámetros se incluyen el ancho de palabra y la profundidad de la FIFO, el número de transacciones a ejecutar, los retardos entre operaciones y la semilla de aleatorización. Adicionalmente, se controlan probabilísticamente los tipos de operación (lectura, escritura, operaciones simultáneas y eventos de reset) mediante la asignación de pesos, lo que permite forzar escenarios específicos como estados de saturación o condiciones de inanición del sistema.

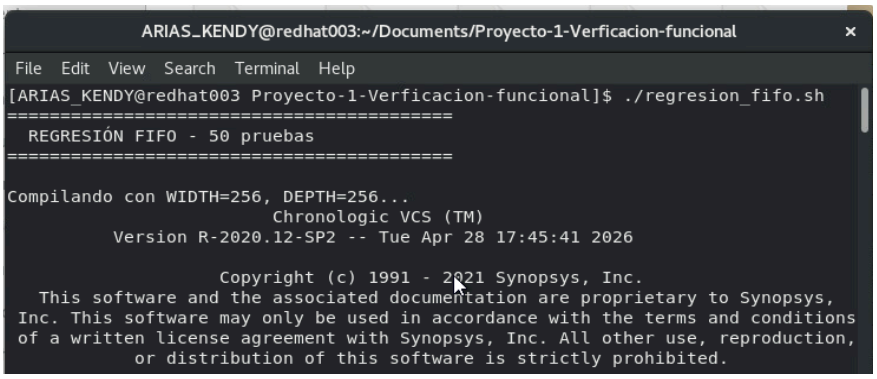

Imagen 11. Ejecución del script "fifo_regression.sh" en la terminal del servidor para el inicio de la regresión automatizada del DUT

El entorno también incorpora la inyección de patrones de datos predefinidos, tales como secuencias constantes (0xAA, 0x55), valores fijos o patrones incrementales, con el objetivo de detectar fallos estructurales en el manejo de datos, incluyendo errores de acoplamiento o condiciones de tipo stuck-at. Asimismo, se establecen estados iniciales controlados de la FIFO, como vacío, lleno o parcialmente ocupado, lo que amplía la cobertura de escenarios evaluados.

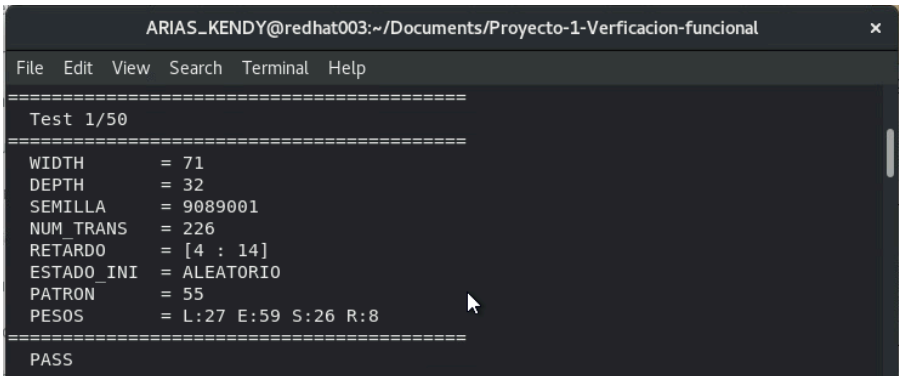

Imagen 12. Ejemplo de test de regresión con configuración aleatoria y resultado exitoso

La validación del sistema se rige por una política de cero tolerancia a errores, en la cual cualquier discrepancia funcional o violación de protocolo resulta en el fallo inmediato de la prueba. El Scoreboard verifica la integridad de los datos y el cumplimiento del protocolo, mientras que el entorno registra automáticamente los escenarios fallidos para su posterior análisis y reproducción.

En caso de fallo, el sistema implementa un mecanismo de depuración completamente reproducible. Cada error es registrado junto con la configuración exacta de la prueba, incluyendo la semilla de aleatorización, lo que permite reconstruir el escenario y analizarlo en detalle mediante herramientas de visualización de señales. Este enfoque es consistente con las prácticas utilizadas en entornos industriales, donde la capacidad de reproducir fallos es fundamental para su resolución. Finalmente, la ejecución del entorno ha sido validada en el servidor institucional, donde el script de regresión permite lanzar múltiples pruebas de forma automatizada con un único comando.

5. Instrucciones de Ejecución

La ejecución del entorno de validación se realiza sobre los servidores institucionales, los cuales ya cuentan con las herramientas de simulación previamente configuradas. Por lo tanto, no es necesario realizar una instalación manual del entorno de Synopsys. El procedimiento consiste en acceder al servidor, transferir la carpeta del proyecto y ejecutar el script de regresión desde la terminal. Una vez ubicado en el directorio correspondiente, la regresión puede iniciarse mediante el siguiente comando:

```
./run_fifo_regression.sh
```

Por defecto, el script ejecuta 50 pruebas automatizadas, generando configuraciones aleatorias en cada iteración. Este comportamiento puede modificarse especificando el número de pruebas como argumento de entrada, según el nivel de cobertura deseado.

Durante la ejecución, el sistema muestra en consola los parámetros de cada prueba, incluyendo el ancho, la profundidad, la semilla de aleatorización y el resultado final (PASS/FAIL). Al finalizar, se presenta un resumen con la cantidad total de pruebas exitosas y fallidas. En caso de detectarse errores, el archivo

failed.txt almacena la configuración exacta de las pruebas fallidas, lo que permite reproducir el escenario específico para su análisis detallado.

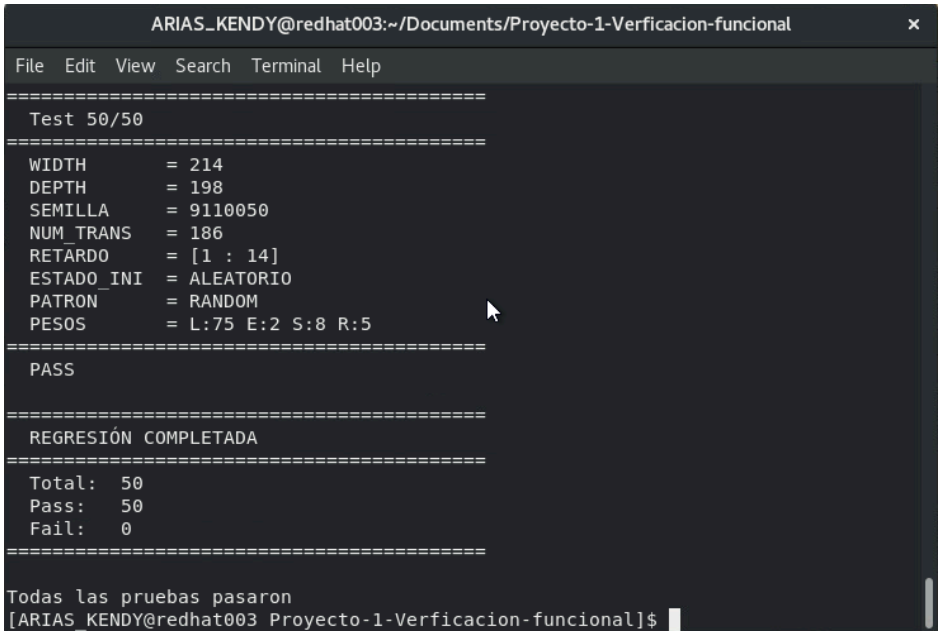

Imagen 13. Resumen final de la regresión automatizada tras la ejecución de 50 pruebas, indicando el estado global del sistema

El código fuente completo del entorno de verificación desarrollado en este proyecto se encuentra disponible en el siguiente repositorio:

https://github.com/Gael-Aguero/Proyecto-1-Verficacion-funcional

6. Conclusiones

El proyecto permitió la modificación y desarrollo de un entorno de verificación completo y flexible para una FIFO genérica, a partir de este trabajo, se pueden destacar los siguientes aspectos:

1. Se implementó un entorno basado en verificación por transacciones y aleatorización controlada (CRV), lo que permitió explorar de manera eficiente el espacio de estados del sistema y detectar condiciones de borde difíciles de identificar manualmente.
2. Se logró desacoplar la arquitectura del testbench mediante el uso de interfaces virtuales y la incorporación de un Monitor independiente, habilitando una verificación de tipo *black-box* basada en el comportamiento real del hardware.
3. La integración de *plusargs* permitió configurar dinámicamente los parámetros de simulación sin necesidad de recompilación, facilitando la generación de múltiples escenarios de prueba con un mismo entorno.
4. Se desarrolló un sistema de regresión automatizada capaz de ejecutar múltiples pruebas con configuraciones aleatorias, incrementando significativamente la cobertura de verificación.
5. El mecanismo de registro de semillas y parámetros de ejecución permitió garantizar la reproducibilidad de fallos, facilitando el proceso de depuración y análisis de errores.

6. El entorno final demuestra ser escalable, reutilizable y alineado con prácticas modernas de verificación de hardware, evidenciando la importancia de la modularidad y la automatización en sistemas digitales complejos.

Bibliografía

- Material del curso *Verificación Funcional de Circuitos Integrados*, Tecnológico de Costa Rica, 2026.
- Apuntes y ejemplos proporcionados por el profesor durante el desarrollo del curso.
- Documentación oficial de SystemVerilog IEEE 1800-2017.
- Recursos en línea sobre verificación funcional y diseño de FIFOs.